



CCM310

Arquitetura de Software e Programação Orientada a Objetos

Profa. Dra. Gabriela Oliveira Biondi

Departamento de Ciência da Computação FEI

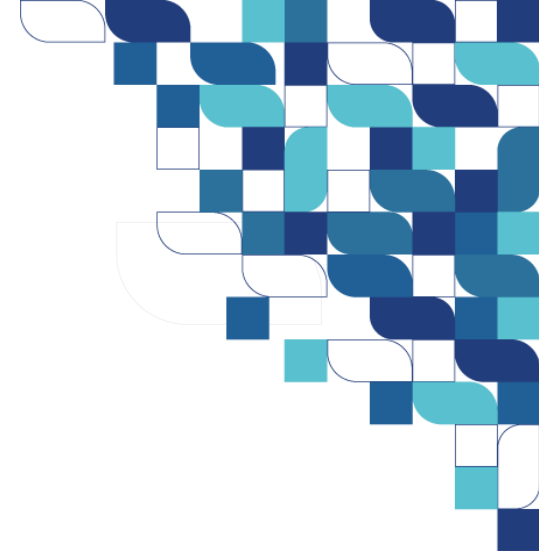


JESUÍTAS BRASIL



Herança

Aula 05



Herança

**É um dos pilares da
Orientação a Objetos!**

Herança

Programação Orientada a Objetos

Abstração

Encapsulamento

Herança

Polimorfismo

Conceitos de Herança

- A ideia é criar uma nova classe a partir de uma classe existente
- É uma forma de **reutilização** de código!
- O conceito de herança se baseia no princípio de que toda codificação mais genérica pode ser transmitida para classes mais específicas
- Relação **é um**

Exemplo de Herança – Mamífero

- Um Cachorro **é um** Mamífero
- Um Gato **é um** Mamífero
- Um Coelho **é um** Mamífero
- Um Humano **é um** Mamífero

Podemos ter uma classe ***Mamífero*** que tem características comuns a todos esses animais!



A classe mamífero pode conter a seguinte característica: ***todos mamam quando filhotes***

Exemplo de Herança - Veículos

- Um Carro **é um** Veículo
- Uma Moto **é um** Veículo
- Um Caminhão **é um** Veículo
- Um Ônibus **é um** Veículo

Podemos ter uma classe ***Veículos*** que tem características comuns a todos esses meios de transporte!



A classe ***Veículo*** pode conter os seguintes atributos:

- ***rodas***
- ***cor***
- ***ano de fabricação***

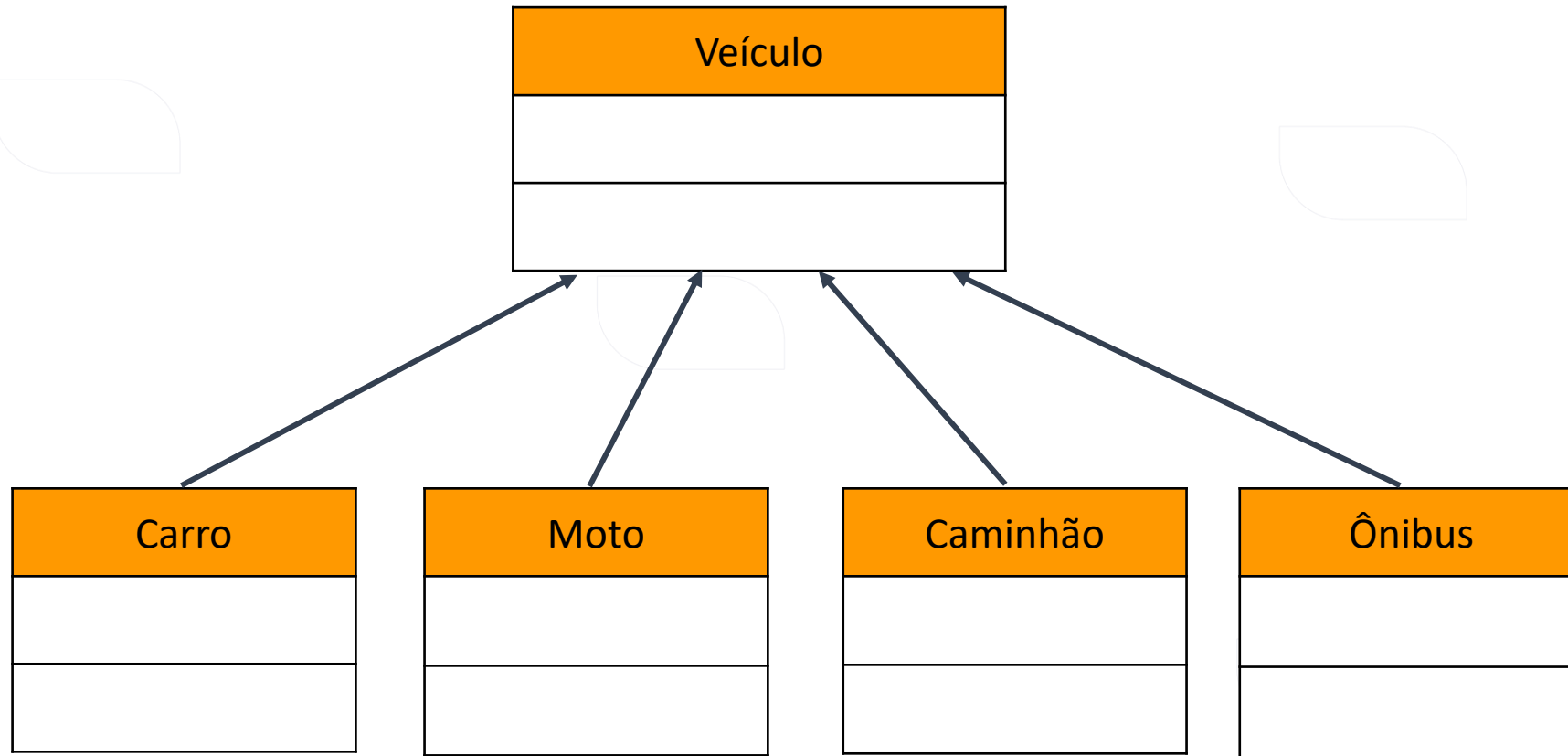
Superclasse e Subclasse

- Considerando os exemplos dados:
 - As classes **Mamífero** e **Veículos** são **superclasses / classes bases**
 - As classes **Cachorro, Gato, Coelho, Humano, Carro, Moto, Caminhão** e **Ônibus** são **subclasses / classes derivadas**
- Superclasses tendem a ser mais genéricas
- Subclasses tendem a ser mais específicas

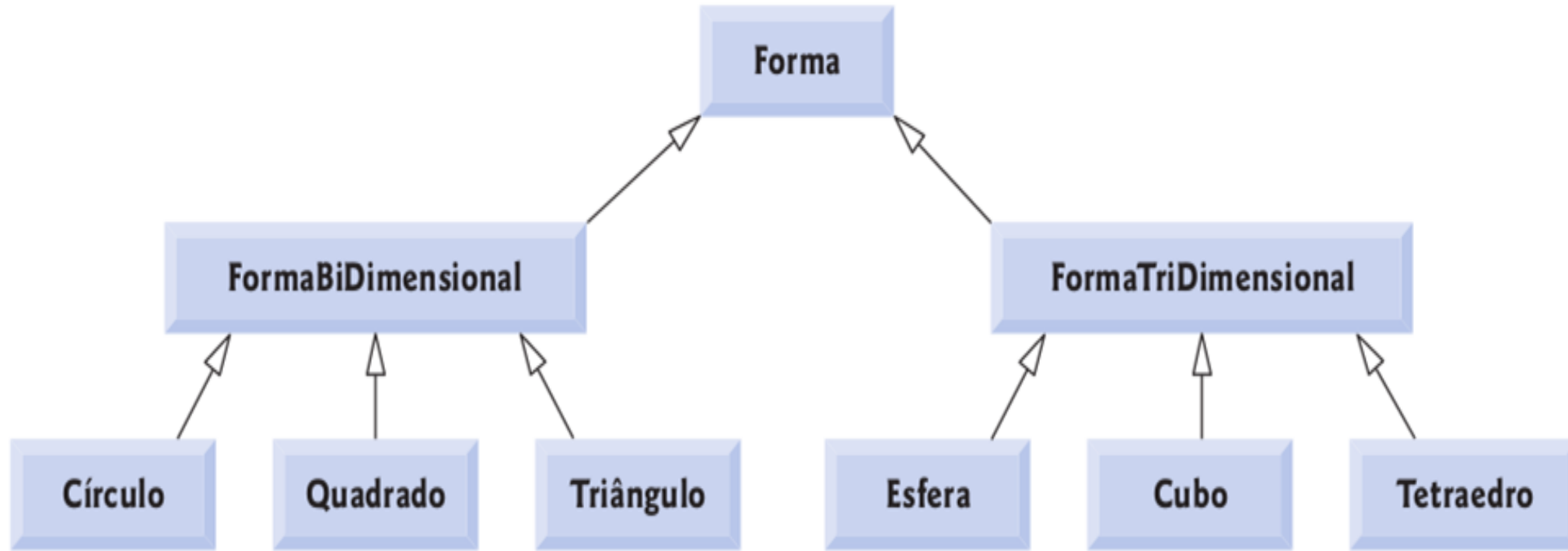
A Herança permite que as Subclasses:

- Herdem os atributos e os métodos da superclasse:
 - atributos e métodos herdados podem ser diretamente utilizados
 - não é preciso escrevê-los novamente
- Definam novos atributos e métodos
- Modifiquem um método definido na superclasse (sobrescrita - override)

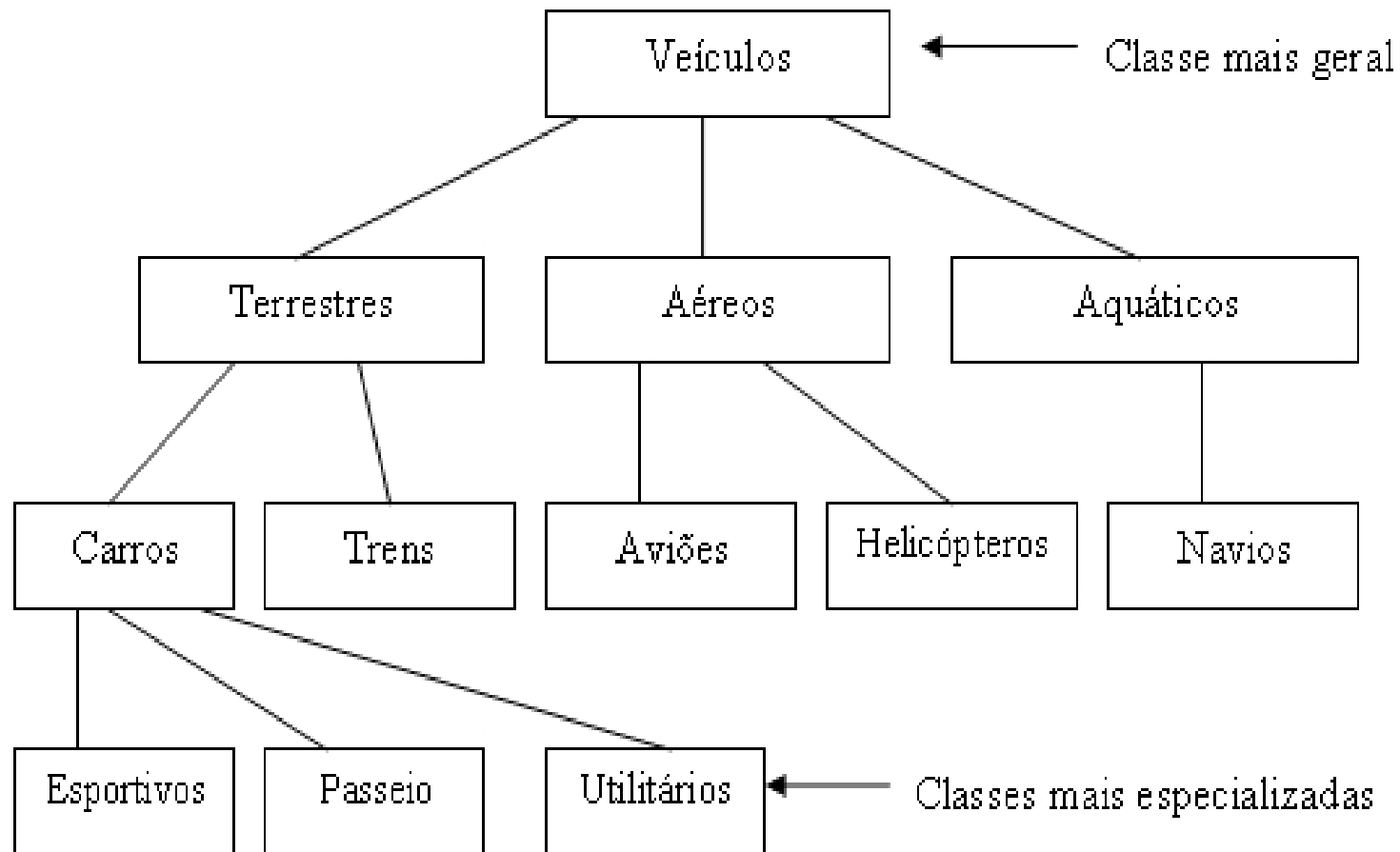
A Herança representada no Diagrama de Classes (UML)



Hierarquia de Classes (UML)



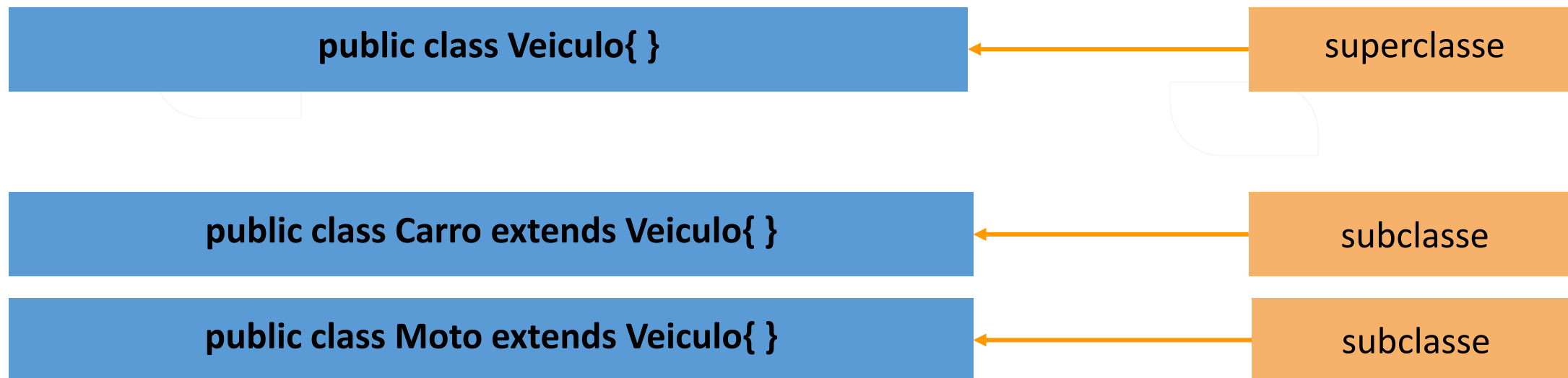
Hierarquia de Classes (UML)



Hierarquia de Classes (UML)

- **Superclasse Direta:** É herdada explicitamente
- **Superclasse Indireta:** É herdada de dois ou mais níveis da hierarquia
- **Herança Simples:** Herda de somente uma classe básica
- **Herança Múltipla:** Herda de múltiplas classes básicas (não existe no Java*)

Herança em Java



Exemplo de Código – Veiculo / Caminhao

```
public class Veiculo {  
    private int rodas, passageiros;  
  
    public int getRodas() {  
        return rodas;  
    }  
  
    public void setRodas(int rodas) {  
        this.rodas = rodas;  
    }  
  
    public int getPassageiros() {  
        return passageiros;  
    }  
  
    public void setPassageiros(int passageiros) {  
        this.passageiros = passageiros;  
    }  
}
```

```
public class Caminhao extends Veiculo {  
    private int carga;  
  
    public int getCarga() {  
        return carga;  
    }  
  
    public void setCarga(int carga) {  
        this.carga = carga;  
    }  
}
```

```
public static void main(String[] args) {  
    Caminhao truck = new Caminhao();  
    truck.setRodas(4);  
    System.out.println( truck.getRodas() );  
}
```

Membros

- **Membros public:** membros public de uma superclasse são acessíveis para os objetos de suas subclasses
- **Membros private:** membros private de uma superclasse são acessíveis somente por dentro da própria classe; não são herdados
- **Membros protected:** membros protected só podem ser acessado em seu próprio pacote (como no package-private) e, além disso, por uma subclasse de sua classe em outro pacote

Construtores e Subclasses

- O construtor da subclasse invoca primeiro o construtor da superclasse
- Os construtores de uma superclasse não são herdados pelas subclasses

Palavra-chave *super*

- Nos casos em que o construtor da superclasse é parametrizado, precisamos chamá-lo explicitamente na subclasse
- A palavra-chave ***super*** permite que chamemos o construtor da superclasse na subclasse

Exemplo de Código – Funcionario / Gerente

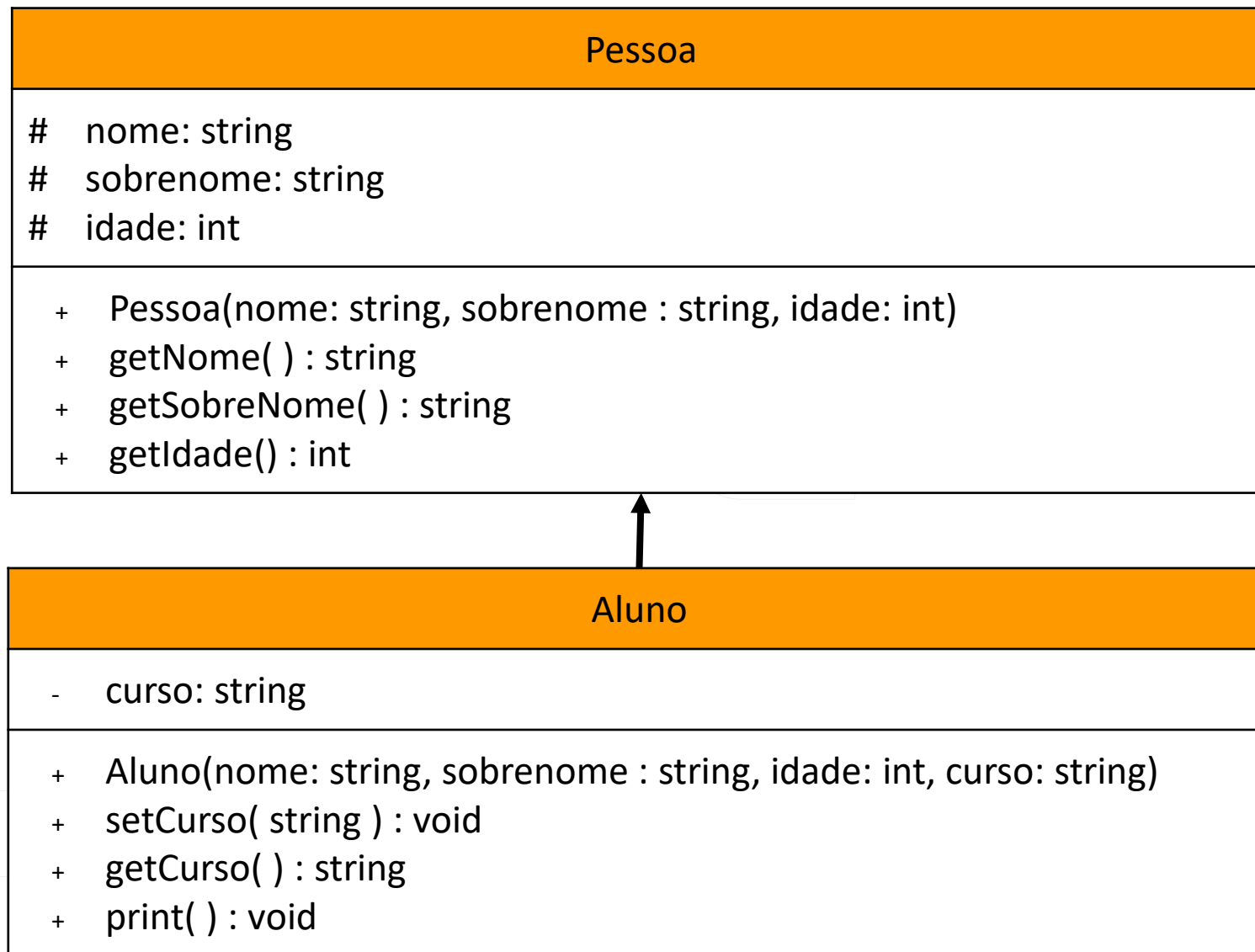
```
1 package classe;
2 public class Funcionario{
3     public Funcionario (String nome, String sobrenome){
4         this.nome = nome;
5         this.sobrenome = sobrenome;
6         System.out.println("Construtor Funcionario");
7     }
8     public String getNome(){
9         return nome;
10    }
11    protected String nome;
12    protected String sobrenome;
13 }
```

```
1 package classe;
2 public class Gerente extends Funcionario{
3     public Gerente (String nome, String sobrenome, int senha){
4         super(nome, sobrenome);
5         this.senha = senha;
6         System.out.println("Construtor Gerente");
7     }
8     public void changeNome(String name){
9         nome = name;
10    }
11    private int senha;
12    private int numeroDeFuncionariosGerenciados;
13 }
```

```
1 package main;
2 import classe.Gerente;
3 class TesteGerente{
4     public static void main(String args[]){
5         Gerente g = new Gerente("fulano", "sicrano", 123456);
6
7         System.out.println(g.getNome());
8         //g.nome="NewName"; // erro pois nome é protected
9         g.changeNome("NewName");
10        System.out.println(g.getNome());
11    }
12 }
```

```
Construtor Funcionario
Construtor Gerente
fulano
NewName
```

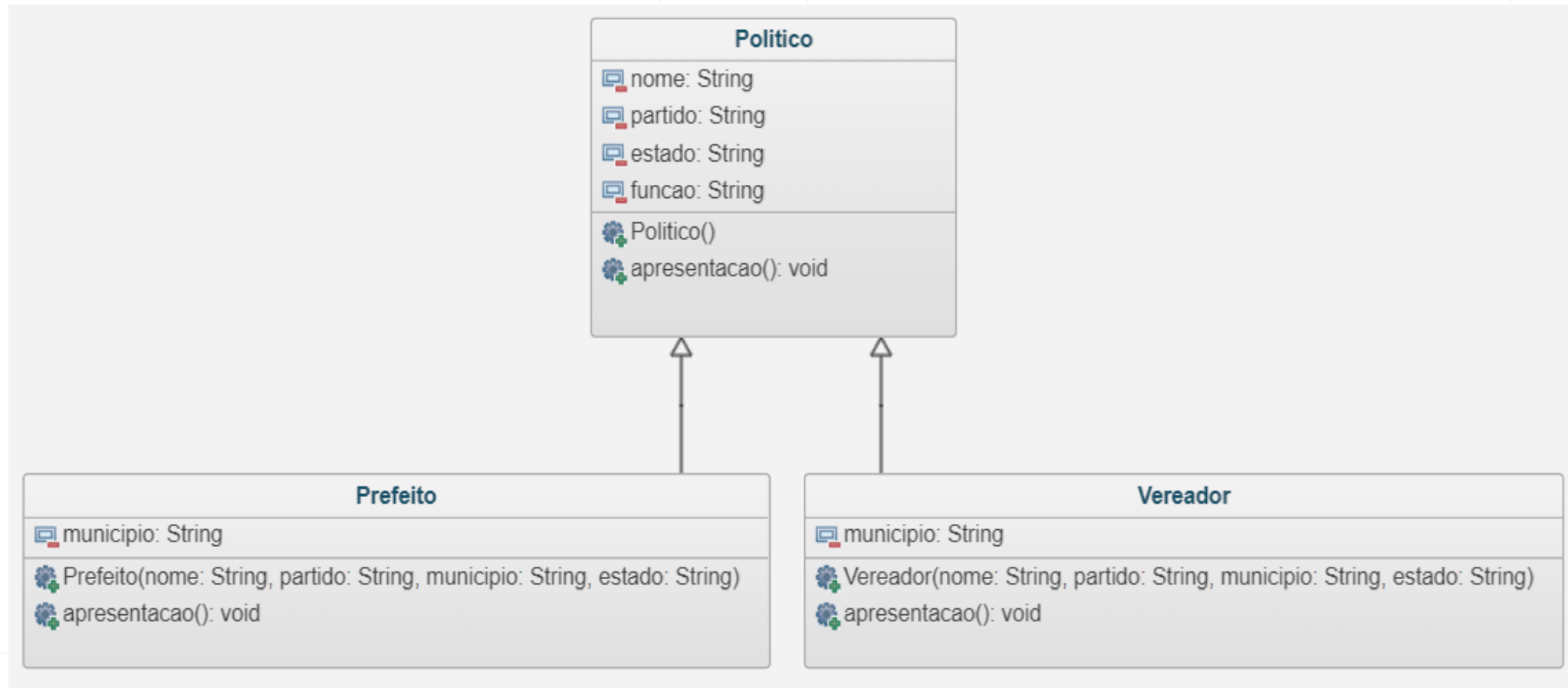
Exercício 1 – Pessoa / Aluno



main():

- obtenha pelo teclado o valor de idade, nome, sobrenome e curso.
- crie um ArrayList de Aluno:
 - Instancie um objeto no ArrayList sem passar argumentos
 - Instancie um outro objeto no ArrayList com o construtor parametrizado
- exiba o conteúdo de todos os Alunos do ArrayList

Exercício 2 – Sistema Político



Exercício 2 – Sistema Político

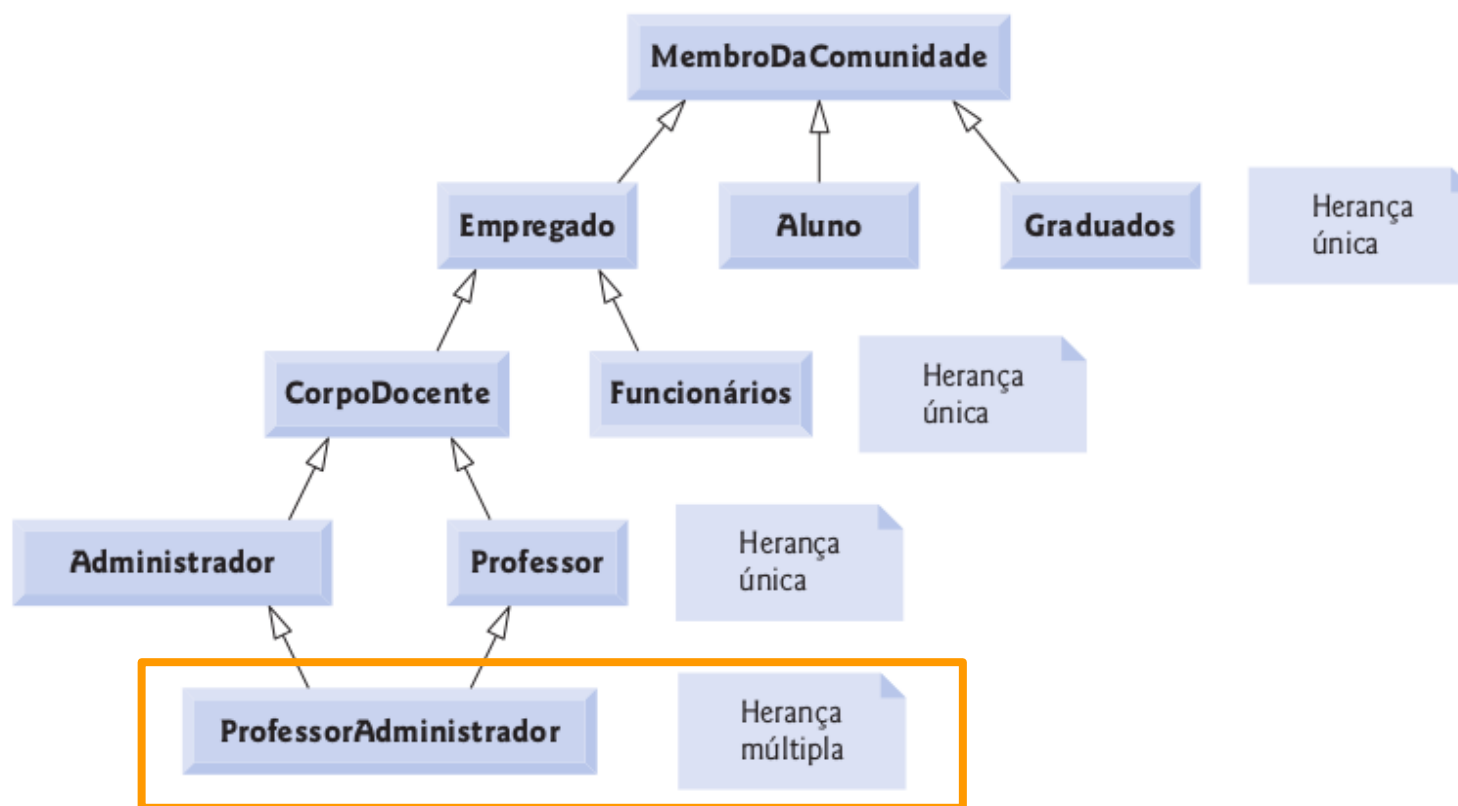
- Implemente a hierarquia de herança do Sistema Político Brasileiro

main():

- Crie 2 ArrayLists: Um para Prefeito e um para Vereador
- Receba todos os dados pelo teclado e instancie:
 - 3 prefeitos
 - 3 vereadores
- Faça com que todos eles se apresentem

Herança Múltipla

- Uma subclasse herda de múltiplas superclasses:



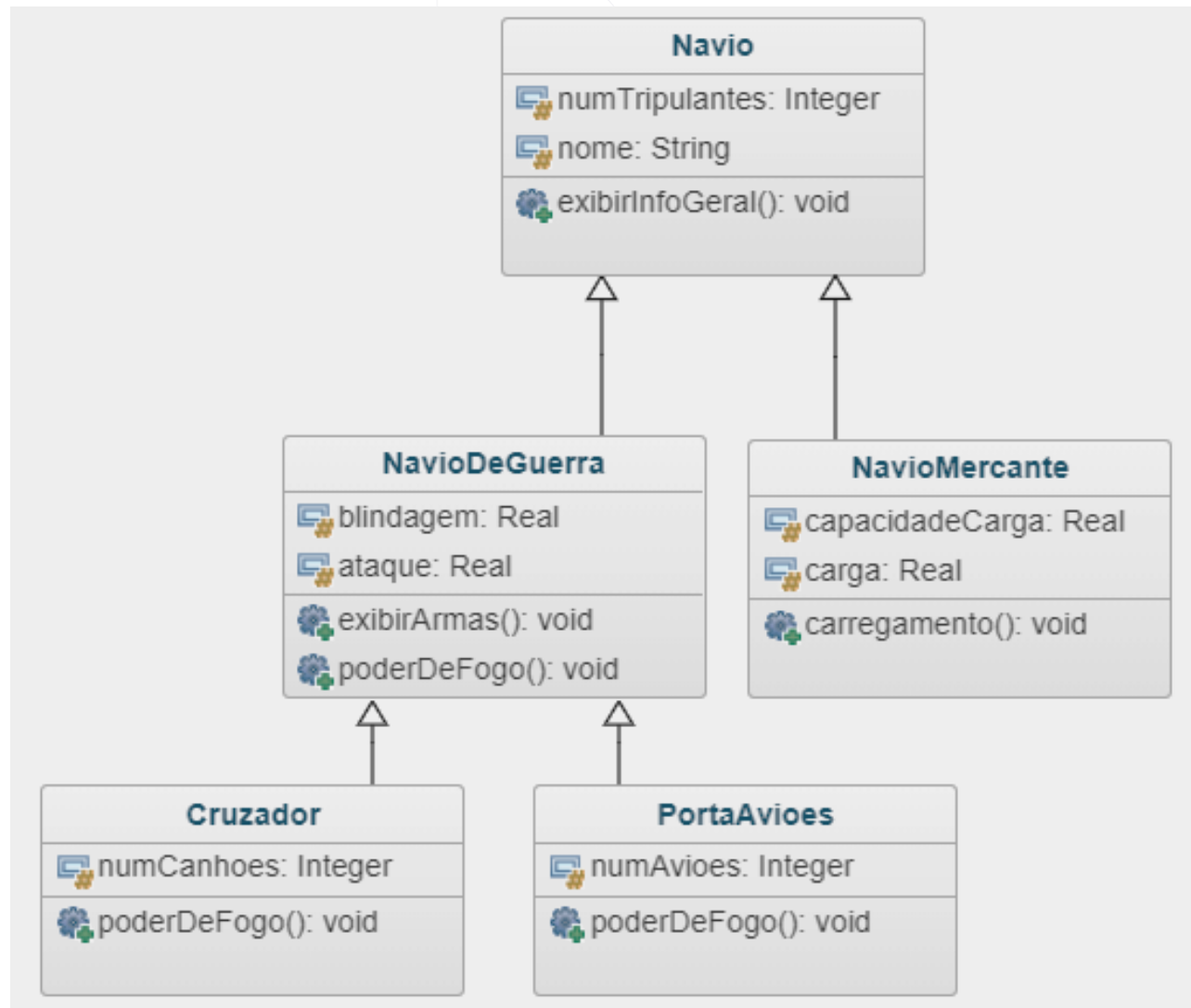
Herança Múltipla

- A herança múltipla pode ser complexa e propensa a erros
- Pode causar ambiguidade!
 - Membros das classes básicas podem ter o mesmo nome
- **Deve ser usada com cautela**

Herança Múltipla

Não tem no Java!
(por enquanto...)

Exercício 3 – Hierarquia Navio



Exercício 3 – Hierarquia Navio

Classe	Método	Descrição
Navio	exibirInfoGeral()	Exibe as informações contidas nos atributos
NavioMercante	carregamento()	Exibe as informações da superclasse e também o volume do Navio ocupado dado por: carga/capacidadeCarga
NavioDeGuerra	poderDeFogo()	Mostra o valor do ataque
NavioDeGuerra	exibirArmas()	Mostra os valores dos campos da superclasse e o valor da blindagem. Chama também o método poderDeFogo()
Cruzador	poderDeFogo()	Mostra o valor calculado por $\text{ataque} * \sqrt{\text{numCanhoes}}$
PortaAvioes	poderDeFogo()	Mostra o valor calculado por $\text{ataque} * \text{numAvioes}^2$

Exercício 3 – Hierarquia Navio

- Implemente a hierarquia de herança de Navio

main():

- Instancie vários objetos da hierarquia Navio.
- Teste todos os objetos, chamando todos os seus métodos

Exercício 4 – Hierarquia de Produtos

- (Adaptado da UFES 2013) Crie uma hierarquia de classes de produtos para uma loja que vende livros, CDs e DVDs. Crie uma superclasse Produto que tenha como atributo nome e tipo. Crie as classes Livro, CD e DVD que herdam de Produto. Sobrescreva o método toString() dessas classes para que imprima:
 - Para livros: nome, preço, autor e gênero;
 - Para CDs: nome, preço, número de faixas e nome de cada faixa;
 - Para DVDs: nome, preço e duração;

Exercício 4 – Hierarquia de Produtos

- Notem que, os atributos que não existirem na classe Produto, devem ser criados nas subclasses.
- Em seguida, crie uma classe Loja com o método main(), que possui um ArrayList que possa guardar os produtos da loja. Adicione 5 produtos diferentes ao ArrayList e, por fim, imprima o conteúdo de todos os produtos.

E a composição?

Herança vs. Composição

Relembrando UML

- **Associação:** Representa um relacionamento entre as classes
- **Agregação:** É uma associação mais específica; relacionamento objeto-todo/objeto-parte (na prática o objeto-todo tem uma referência/ponteiro ao objeto-parte)
- **Composição:** É uma agregação mais forte, objetos-parte não existem sem o objeto-todo (na prática o objeto-parte é instanciado dentro do objeto-todo)

Herança vs. Composição

- Composição e Herança são dois mecanismos para reutilizar funcionalidades
- Na herança temos o conceito de Classe Base/Superclasse que é a classe que foi herdada pelas Classes Derivadas/Subclasses
- A composição por sua vez estende uma classe pela delegação de trabalho para outro objeto

Herança vs. Composição

- **é um** versus **tem um**
- **é um** (Herança)
 - O objeto da classe derivada pode ser tratado com objeto da classe básica.
 - Exemplo: O carro é um veículo
- **tem um** (Composição)
 - O objeto contém um ou mais objetos de outras classes como membros.
 - Exemplo: O carro tem uma direção

Vantagens e Desvantagens da Herança



- A herança é vista diretamente no código
- Permite que menos linhas de código sejam digitadas



- Entre os problemas da Herança está o fraco encapsulamento entre superclasses e subclasses e o forte acoplamento entre elas: ao mudar uma superclasse, todas as subclasses podem ser afetadas

Vantagens e Desvantagens da Composição



- Os objetos que foram instanciados e estão contidos na classe que os instanciou são acessados somente através de sua interface seguindo assim o princípio de programar para uma interface e não para uma implementação

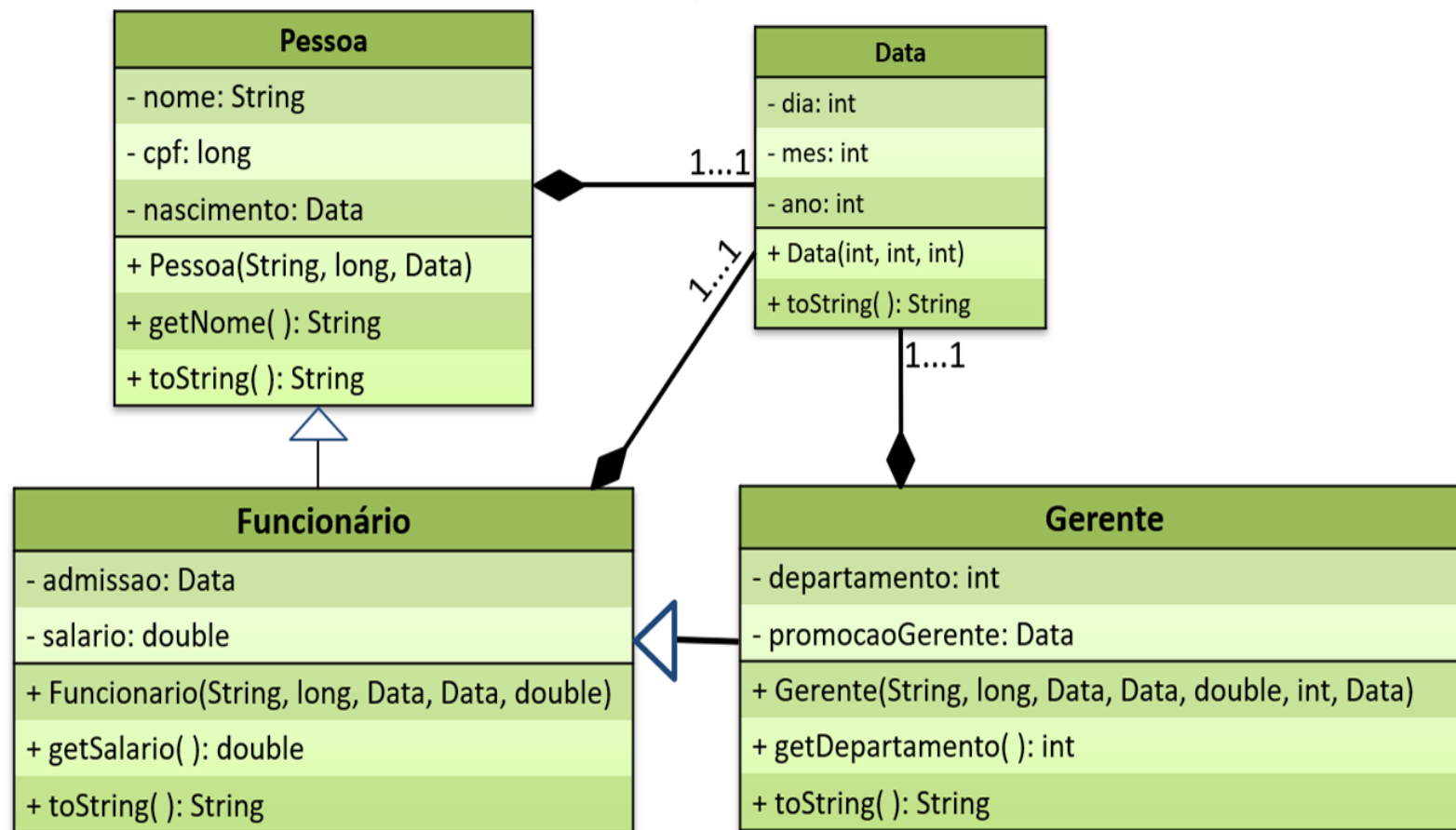


- A grande desvantagem é que um software muito dinâmico e parametrizado tem seu entendimento mais complicado
- Muitas linhas de código precisam ser reescritas

Qual usar?

- Existem várias discussões, mas a grande maioria sugere o uso de Composição ao invés de Herança
- De forma geral, utilizamos herança quando temos a relação é um e composição quando temos a relação tem um

Exercício 5 – Composição + Herança



Atenção para o que é **herança** e o que é **composição**!

Exercício 5 – Composição + Herança

- No método main instancie alguns funcionários e alguns gerentes em dois ArrayLists distintos.
- Chame, por meio dos objetos que estão dentro dos ArrayLists, os métodos de ambos para testar.

Exercício 6 – Herança de Account

- (Deitel 12.10) (Hierarquia de herança de Account) Crie uma hierarquia de herança que um banco possa utilizar para representar as contas bancárias dos clientes. Todos os clientes nesse banco podem depositar (isto é, creditar) dinheiro em suas contas e retirar (isto é, debitar) o dinheiro delas. Há também tipos mais específicos de contas. As contas de poupança, por exemplo, recebem juros pelo dinheiro depositado nelas. As contas correntes, por outro lado, cobram uma taxa por transação (isto é, crédito ou débito).

Exercício 6 – Herança de Account

- Crie uma hierarquia de herança contendo a superclasse Account e as subclasses SavingsAccount (Poupança) e CheckingAccount (Conta Corrente) que herdam da classe Account. A superclasse Account deve incluir um membro de dados do tipo double para representar o saldo da conta.
- A classe deve fornecer um construtor que recebe um saldo inicial e o utiliza para inicializar o membro de dados. O construtor deve validar o saldo inicial para assegurar que ele é maior que ou igual a 0.0. Caso contrário, o saldo deve ser configurado como 0.0 e o construtor deve exibir uma mensagem de erro, indicando que o saldo inicial era inválido.

Exercício 6 – Herança de Account

- A classe deve fornecer três métodos. O método `credit` deve adicionar uma quantia ao saldo atual. O método `debit` deve retirar dinheiro de `Account` e assegurar que o valor do débito não exceda o saldo de `Account`. Se exceder, o saldo deve permanecer inalterado e a função deve imprimir a mensagem “Saldo insuficiente”. O método `getBalance` deve retornar o saldo atual. A subclasse `SavingsAccount` deve herdar a funcionalidade de uma `Account`, mas também incluir um membro de dados do tipo `double` para indicar a taxa de juros (porcentagem) atribuída à `Account`.

Exercício 6 – Herança de Account

- O construtor SavingsAccount deve receber o saldo inicial, bem como um valor inicial para a taxa de juros de SavingsAccount. SavingsAccount deve fornecer um método public calculateInterest que retorna um double para indicar os juros auferidos por uma conta. O método calculateInterest deve determinar esse valor multiplicando a taxa de juros pelo saldo da conta.
- [Nota: SavingsAccount deve herdar as funções-membro credit e debit exatamente como são sem redefini-las.]

Exercício 6 – Herança de Account

- A subclasse CheckingAccount deve herdar da classe básica Account e incluir um membro adicional de dados do tipo double que representa a taxa cobrada por transação. O construtor CheckingAccount deve receber o saldo inicial, bem como um parâmetro que indica o valor de uma taxa. A classe CheckingAccount deve redefinir as funções-membro credit e debit para que subtraíam a taxa do saldo da conta sempre que qualquer uma das transações for realizada com sucesso.
- As versões CheckingAccount dessas funções devem invocar a versão Account da classe básica para realizar as atualizações de saldo de uma conta. A função debit de CheckingAccount deve cobrar uma taxa somente se o dinheiro for realmente retirado (isto é, o valor do débito não exceder ao do saldo da conta).

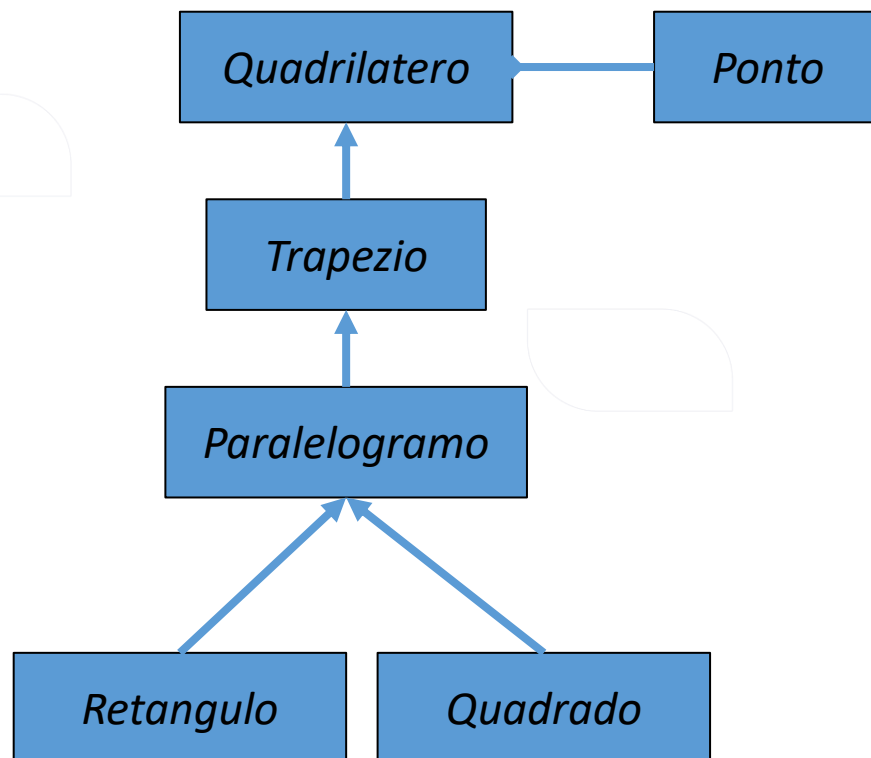
Exercício 6 – Herança de Account

- Dica: Defina a função debit de Account para que ela retorne um bool indicando se houve retirada de dinheiro. Em seguida, utilize o valor de retorno para determinar se uma taxa deve ser cobrada.

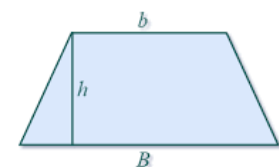
Exercício 7 – Hierarquia Quadrilatero

- (Deitel 9.8) Escreva uma hierarquia de herança para as classes Quadrilatero, Trapezio, Paralelogramo, Retangulo e Quadrado. Use Quadrilatero como superclasse da hierarquia. Crie e use uma classe Ponto para representar os pontos (x, y) de cada forma. Faça a hierarquia o mais profunda (isto é, com muitos níveis) possível. Especifique as variáveis de instância e os métodos para cada classe. As variáveis de instância private de Quadrilatero devem ser Pontos para os quatro pontos que delimitam o quadrilátero. Escreva um programa que instancia objetos de suas classes e gera saída da área de cada objeto (exceto Quadrilatero). A entrada será feita com a posição de 4 pontos (x, y) .

Exercício 7 – Hierarquia Quadrilatero



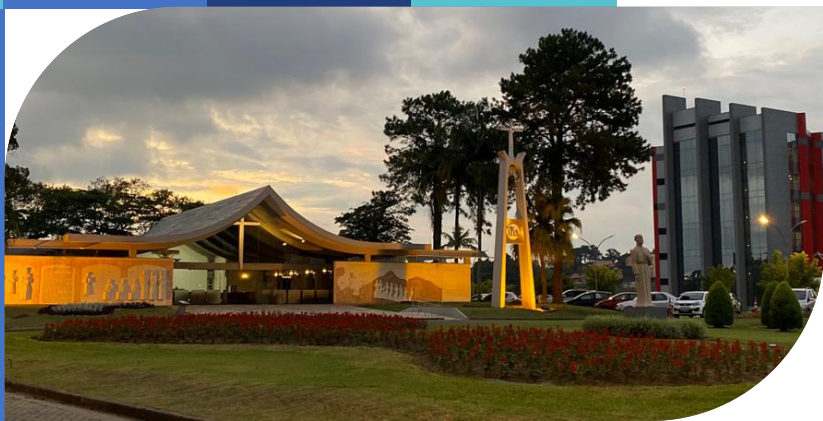
O cálculo de área do trapézio serve também para o cálculo de área de todas as classes que herdam de Trapézio


$$A = \frac{(B + b)h}{2}$$



**Obrigada pela sua
participação, nos vemos na
próxima aula!**

:)



FEJ.
edu.br



JESUÍTAS BRASIL